Your Quantization Format Is Not Free: Same-Size, Same-Label GGUF Files Decode Very Differently on Edge CPUs

Manu Nicholas Jacob
University of Massachusetts Amherst
Amherst, MA, USA
manunicholasjacob@gmail.com

Abstract

On-device LLM deployment is guided by a byte-streaming mental model: decode reads every weight per token, so throughput and energy are set by model bytes, and a quantization format is chosen by accuracy per byte. We report controlled measurements, on three CPU microarchitectures spanning the edge spectrum (Cortex-A76, Gracemont, Golden Cove), showing that the format *label* is a poor predictor of what a GGUF file actually costs. Three findings. (1) **Provenance:** FP16-sourced files decode up to 38% faster than Q8-requantized files carrying the same format label and nominal bit width, because quantization provenance silently changes per-tensor layouts, and at sub-billion scale the output head alone is over 40% of the bytes streamed per token. (2) **The default is taxed where it is least expected:** normalized by bytes actually streamed, Q4_K_M, the community-default format, falls 16% (A76), 19% (Gracemont), and 28% (Golden Cove) below the streaming envelope at 0.5B, where on a Raspberry Pi 5's PMIC it costs 44% more energy per token than an i-quant of near-identical quality; at 1.5B it recovers, exactly where both the quantizer's fallback types and the runtime's kernel coverage recover. (3) **Rankings do not transfer:** the fastest 4-bit format on P-cores (Q4_0) is mid-pack on E-cores, where IQ4_NL wins by 19%; on the A76 the whole 4-bit class converges. The pattern is consistent with a two-term cost model, decode time as the maximum of a streaming term and a per-(format, kernel, core) compute term, and with the uneven coverage of architecture-specific repacked kernels in today's runtimes. The practical conclusions are uncomfortable: accuracy-per-byte rankings assume kernel parity that does not exist, benchmark results do not transfer across artifact provenances, and the right format is a property of the device, not the model.

1 Introduction

Small LLMs now run locally on phones, wearables, and single-board computers, and on most of them decode executes on the CPU [1, 10, 15]. Deployment folklore rests on a byte-streaming model: each generated token must read essentially all weights, weights dwarf cache, therefore

$$\text{tok/s} \approx B_{\text{eff}}/\text{bytes}, \tag{1}$$

with B_{eff} a bandwidth constant of the device [6, 11, 16]. Under this model the only property of a quantization format that matters at runtime is its size, so formats are ranked by accuracy per byte [4] and the ranking is assumed to hold on every device. The `llama.cpp` ecosystem [5], the de facto runtime for CPU inference at the edge, encodes this assumption in its defaults: model cards recommend Q4_K_M as the balanced choice, and users pick a GGUF file by its type label.

This paper measures that assumption and finds it wrong in three separate ways. We benchmark eight GGUF formats of one model, quantized from the FP16 source exactly as public repositories ship them, on three CPU microarchitectures that bracket the edge: the Cortex-A76 of a Raspberry Pi 5 (the same core class as the big cores of hundreds of millions of mid-tier phones, where measured decode runs 2 to 4 tok/s [15]), and the Gracemont E-cores and Golden Cove P-cores of one laptop SoC, pinned separately so core type varies while the memory system stays fixed. Throughput is measured unperturbed; Pi package energy comes from the on-board PMIC in a separate identical run; quality is perplexity on a held-out corpus from the same artifacts.

Finding 1: the label is not the layout. A GGUF file's performance-relevant content is its per-tensor type map, and that map depends on where the file came from. Requantizing the same model from a Q8_0 intermediate instead of FP16 (a common practice, and one `llama-quantize` performs without warning) leaves the same label on a file with a different per-tensor type map; the FP16-sourced i-quant variants decode up to *38% faster* than their Q8-requantized counterparts on the A76. The mechanism is composition: at 0.5B scale the output head is over 40% of the bytes streamed per token, and its type, which the label does not describe, is set silently by provenance.

Finding 2: the default format is off the envelope on every core, at the scale most edge deployments use. Once throughput is normalized by *streamed* bytes (repeating layers plus output head, not file size), most formats sit on

a common streaming envelope per platform, but at 0.5B Q4_K_M sits 16 to 28% below it on every core we tested, and on the Pi it costs 44% more energy per token than IQ4_XS at a quality difference of 0.6 perplexity points. The deficit has two causes: at 0.5B the 256-divisibility fallback inflates Q4_K_M to 6.2 BPW (more bytes streamed than the label implies), and the runtime's repack-kernel coverage for its actual tensor types is only 12% on x86. At 1.5B both resolve (the wider embedding dimension passes the 256 threshold, and k-quant coverage increases), and Q4_K_M rejoins the envelope: neither the tax nor its absence is knowable from the label.

Finding 3: format rankings do not transfer across cores. Q4_0 leads Golden Cove at 2 threads by 22% and is mid-pack on Gracemont, where IQ4_NL wins at 4 threads by 19%; on the A76 the entire 4-bit class converges to within 4%. A ranking measured on the developer's laptop is silently wrong on the deployment target, in either direction.

We show the pattern is what a two-term cost model predicts: decode time is the maximum of a streaming term and a compute term whose coefficient belongs not to the format alone but to the (format, kernel, core) triple, and we connect the observed winners to the uneven per-architecture coverage of repacked kernels in the runtime. We close with what this means for benchmarks, runtimes, and deployers.

2 Measurement design

Artifacts. Qwen2.5-0.5B-Instruct [12], converted with the `llama-quantize` tool (build `d73c1d6`) from the official FP16 GGUF into Q4_0, IQ4_NL, IQ4_XS, Q3_K_M, Q2_K, Q4_K_M, Q6_K, and Q8_0. Our files match the official repository's published sizes byte-for-byte where both exist, so these are the artifacts users actually download. A second, provenance arm requantizes the same targets from a Q8_0 intermediate. At 0.5B scale the embedding dimension (896) is not divisible by 256, triggering silent per-tensor fallbacks in `llama-quantize`: Q4_K_M loses 133 of 159 weight tensors to higher-precision fallback types (q4_K→q5_0, q6_K→q8_0), inflating its effective bit rate from the nominal 4.5 to approximately 6.2 BPW; the simpler formats (Q4_0, IQ4_NL, IQ4_XS) have no 256-column constraint and quantize without fallback. The tool emits per-tensor warnings but does not refuse or prominently flag the aggregate inflation. All artifacts, raw measurement records, quantization logs, and scripts accompany the paper.[1]

Streamed bytes, not file bytes. Decode reads the repeating layers and the output head every token; of the embedding table it reads one row. File size therefore mismeasures the traffic that Eq. 1 is about, and the mismatch is large and format-dependent: at 0.5B scale the embedding and output tensors are a third of the file, their types vary by format and provenance (the output head falls back to q8_0 from the q6_K target in FP16-sourced files, producing a 138 MiB tensor; in Q8-requantized files the embedding and head share a single tied Q8_0 tensor), and the head alone is 138 MiB of the roughly 330 MiB streamed per token. We parse every artifact's tensor map and normalize throughput by streamed bytes throughout. This single step reclassifies several published-style comparisons: our 4-bit files span 403 to 464 MiB on disk but 317 to 374 MiB streamed.

[1]Public artifact repository linked at submission.

Table 1: Canonical (FP16-sourced) artifacts on the Pi 5 (A76, 2 threads): decode rate, streamed-byte throughput, PMIC energy, perplexity. Envelope ≈ 11.7 to 12.0 GB/s.

Format	MiB_{str}	tok/s	GB/s_{str}	mJ/tok	ppl
Q4_0	330	**34.7**	12.0	**142**	21.71
IQ4_XS	330	34.2	11.8	143	20.70
IQ4_NL	332	34.2	11.9	147	20.70
Q3_K_M	334	33.4	11.7	147	21.04
Q2_K	317	31.8	10.6	155	21.83
Q4_K_M	374	25.7	**10.1**	**206**	20.12
Q6_K	477	23.4	11.7	204	19.52
Q8_0	501	22.2	11.7	216	19.52

Platforms and protocol. (i) Raspberry Pi 5 (4×Cortex-A76, 2.4 GHz, 2 GB LPDDR4X, 64-bit Linux, native build with dot-product extensions); (ii) i7-12700H, with `llama-bench` pinned by affinity mask to either the 8 Gracemont E-cores or the 6 Golden Cove P-cores (Alder Lake's hybrid design gives two microarchitectures on one memory system; prior work on this machine verified the pinning by per-core throughput). Decode throughput is `llama-bench` generation rate (`-p 128 -n 128`, 5 repetitions; per-format standard deviation on the Pi is at most 0.34 tok/s). Throughput runs are unperturbed; a reverse-order control run with per-bench temperature, frequency, and throttle-flag logging confirmed no thermal throttling (`throttled=0x0`, 2.4 GHz sustained, board temperature 46–55 °C) and reproduced the canonical ordering within 2% for the formats unaffected by board contention. Pi package power (PMIC rail sum at 10 Hz, uncalibrated, so relative comparisons only) is collected in a separate identical run, following the two-run practice of prior instrumented studies. Perplexity uses 100×512-token chunks of a public-domain corpus; format deltas are paired on identical text.

3 Results

The envelope, and who is off it (Table 1, Fig. 1). On the A76, six of eight formats sit within 3% of an 11.7 to 12.0 GB/s

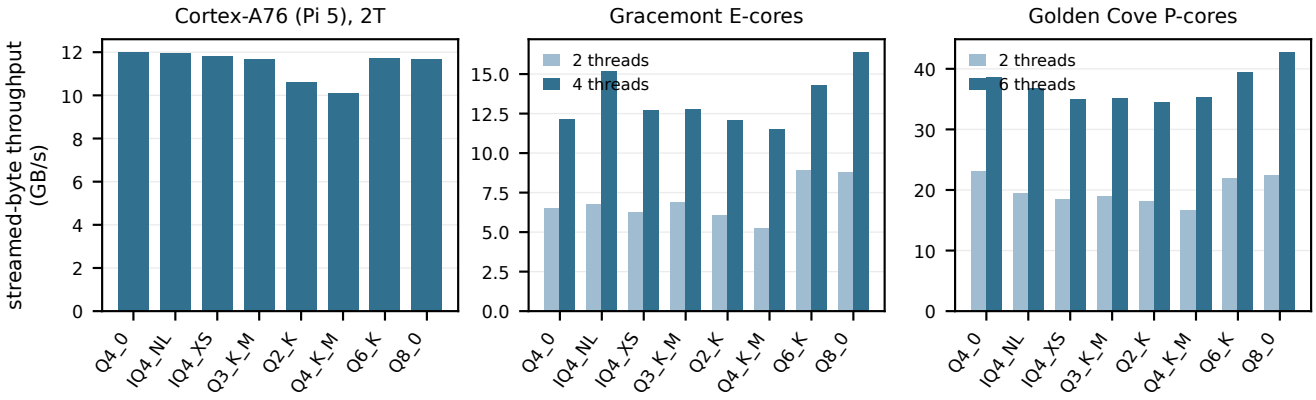

Figure 1: Streamed-byte decode throughput per format on three microarchitectures (canonical artifacts). Under byte-streaming all bars within a panel would be equal. The A76 converges except Q2_K and Q4_K_M; Golden Cove at 2 threads favors Q4_0 and Q8_0 by 15 to 25%; Gracemont inverts the P-core ranking, with IQ4_NL on top at 4 threads. Q4_K_M is at or near the bottom of every panel.

streamed-byte envelope, consistent with this board's previously measured effective decode bandwidth. Two do not: Q2_K (down 11%) and Q4_K_M (down 16%). On Golden Cove at 2 threads the envelope is about 23 GB/s and only Q4_0, Q6_K, and Q8_0 reach it; the i-quants sit 15 to 20% below and Q4_K_M 28% below. At 6 threads the spread compresses toward the memory ceiling. Gracemont shows a third ordering: at 4 threads IQ4_NL is the *fastest* format on the machine (15.2 GB/s streamed against Q4_0's 12.2), the reverse of the P-core ranking on the same silicon, same DRAM, same binary.

Energy (A76, PMIC, uncalibrated). Energy per token tracks time, not power: package power varies only 4.4 to 5.3 W across formats while per-token energy spans 142 to 216 mJ. The consequential comparison is at matched quality class: Q4_K_M spends 206 mJ/token where IQ4_XS spends 143 mJ for a 0.58-point perplexity difference, a 44% energy premium; and Q6_K delivers a 0.60-point *better* perplexity than Q4_K_M at equal energy (204 vs 206 mJ). On this board the default format is Pareto-dominated for every objective except a small quality edge over the 4-bit class.

Quality completes a three-way trade. Perplexity ranks IQ4_XS/IQ4_NL (20.70) ahead of Q3_K_M (21.04) and Q4_0 (21.71) within the 4-bit class, with Q4_K_M between them (20.12). On Golden Cove at low thread count, Q4_0's speed lead therefore prices quality against latency; on the A76, where the class converges in speed, IQ4_XS is simply the sweet spot (near-best energy, best 4-bit quality). The accuracy-per-byte ranking that motivates i-quants is realized on exactly one of our three cores at deployment-typical thread counts.

Provenance: same label, 38% apart. Our provenance arm requantized the same eight targets from a Q8_0 intermediate. The resulting files carry identical labels but their tensor maps differ: the tied embedding/head tensor stays Q8_0, and several tensors take fallback types. On the A76 the FP16-sourced i-quant files decode at 34.2 tok/s against the Q8-requantized files' 24.7 to 26.0 tok/s: the canonical files are up to 38% faster than counterparts a user cannot distinguish by name or nominal bit width. Any benchmark that does not pin artifact provenance is measuring an unknown.

Threads move the tax, as a compute term should. On Gracemont the matched-size 4-bit class spreads 32% at 2 threads and 8% at 4; on Golden Cove the 2-thread spread of 25% compresses to 11% at 6. A cost that shrinks as cores are added while traffic is constant is a compute-side cost. An independently collected campaign on the same machine (different session and harness, published dataset [9]) reproduces the high-thread ordering at 8 threads.

Attribution: the taxed formats compute, the fast ones stall. Table 2 instruments the mechanism with `perf` counters and the runtime's own load logs. Q4_K_M retires **0.439 billion instructions per token, 2.8× Q4_0's 0.158**, and runs at IPC 2.17: the cores are not waiting on DRAM, they are busy unpacking superblocks. The envelope formats show the opposite signature (Q8_0 at IPC 0.64, the classic memory-stall profile). Instructions per token, an artifact-plus-kernel property measurable in one run, predicts the streamed-bandwidth deficit across all eight formats.

Out of sample: coverage moves with scale, and speed follows. The runtime's repack coverage is per-tensor, so it changes with model size: at 1.5B, Q4_K_M's repacked fraction jumps from 12% to a majority of its weights (and the

Table 2: Kernel attribution on the A76 (canonical artifacts, 2 threads): bytes the runtime repacks into architecture-tuned kernels at load, retired instructions per generated token, and IPC. Envelope formats are memory-stalled (low IPC); the taxed formats are compute-busy (high IPC).

Format	repacked/streamed MiB	Ginst/tok	IPC
Q8_0	501 / 501	**0.140**	**0.64**
Q4_0	330 / 330	0.158	1.09
Q6_K	476 / 477	0.183	0.86
IQ4_NL	330 / 332	0.192	1.31
Q3_K_M	319 / 334	0.194	1.29
IQ4_XS	281 / 330	0.217	1.47
Q2_K	272 / 317	0.240	1.50
Q4_K_M	208 / 374	**0.439**	**2.17**

wider embedding dimension eliminates the 256-divisibility fallback), while IQ4_XS's coverage drops from 70% to zero. This yields a prediction, made before the 1.5B benchmarks ran: Q4_K_M should recover, IQ4_XS should become the laggard. Both hold, on both x86 core types. At 1.5B on 2 P-core threads the 4-bit file-byte implied-bandwidth ordering (streamed-byte tensor maps not parsed at this scale) becomes Q4_K_M (23.4 GB/s) ≈ Q4_0 (22.8) > IQ4_NL (21.7) > IQ4_XS (18.7) ≈ Q3_K_M (18.0), a reshuffle of the 0.5B ordering that tracks the coverage shift format by format. The A76 agrees where its own coverage says it should: at 1.5B the quartet holds a common envelope within 10% and Q4_K_M rejoins it, but Q3_K_M drops roughly 28%, mirroring its coverage loss, while IQ4_XS, the zero-coverage x86 laggard, stays on the A76 envelope where the ARM `iq4_nl_4x4` repack still covers it. One artifact, slowest 4-bit choice on one ISA and fastest on the other. The format tax is not a fixed property of a format; it follows the kernels.

4 A two-term model, and where the constants really live

The observations fit

$$t_{\text{tok}} = \max\left(\frac{\text{bytes}_{\text{str}}}{B_{\text{eff}}},\ \frac{\text{bytes}_{\text{str}} \cdot c_{f,k}}{C}\right), \tag{2}$$

with C the compute throughput of the thread configuration and $c_{f,k}$ the unpack-and-multiply cost per streamed byte. The essential empirical point is that $c_{f,k}$ is *not a property of the format*: it belongs to the (format, kernel, core) triple. At 0.5B scale, Q4_K_M's deficit is compounded by the 256-divisibility fallback that inflates its actual tensor types beyond the label's bit rate, making the "format" it runs a different artifact from the one its name implies. We verify the kernel dependence three ways rather than assert it. First, the runtime says so: `llama.cpp`'s load logs report exactly which tensors it repacks into architecture-tuned kernels (the repack mechanism and per-format kernel coverage are documented in project PRs and developer discussions [5]), and on x86 the 4-bit class's coverage (Q4_0 100% of repeating bytes, IQ4_NL 94%, Q3_K_M 91%, IQ4_XS 70%, Q4_K_M **12%**) matches the low-thread speed ordering, while zero-coverage Q6_K and Q8_0 still reach the envelope because their plain kernels are cheap. Second, the hardware says so: instructions per token and IPC (Table 2) put the taxed formats on the compute branch of Eq. 2 and the envelope formats on the memory-stall branch. Third, the mechanism predicts out of sample: when coverage shifts with model scale, the speed ordering reshuffles with it (§3). Formats with a well-covered, cheap kernel on a given core ride the streaming envelope; formats without one pay a compute term that shrinks with added threads.

Eq. 2 also retro-predicts results that prior instrumented studies reported but did not explain: energy inversions in which a smaller format costs more than a larger one on a Raspberry Pi 4 [8], and the latency increases below 4 bits that motivated lookup-table kernel projects [13, 14]. Both are the right branch of the max binding.

5 What should change

For benchmarks. Mobile and edge LLM evaluations [1, 2, 10, 15] report format as a label and size as file size. Both are underspecified: results should record the per-tensor type map (or artifact hash and provenance) and normalize by streamed bytes. Without this, numbers do not replicate across artifacts that look identical, as our 38% provenance gap shows.

For runtimes. Kernel coverage is a first-class performance property and today it is invisible. A runtime that knows $c_{f,k}$ for its own kernels on the local core could pick the layout at install time; our data says the win from doing so is up to a third of throughput and 44% of energy at essentially no quality cost. Until then, model cards recommending one format for all devices are wrong on some of them by construction.

For deployers. On these three cores the measured guidance is concrete: on A76-class devices choose IQ4_XS (best 4-bit quality at envelope speed and near-minimum energy); on Golden-Cove-class cores at low thread budgets choose Q4_0 if latency dominates and IQ4 if quality does; on Gracemont-class cores IQ4_NL wins outright at 4 threads; on all of them, do not default to Q4_K_M without checking, and never trust a benchmark run on a differently-sourced file. More generally: measure on the target, because the ranking will not transfer.

6 Related work

Byte-streaming analyses of decode [2, 6, 11, 16] and mobile measurement studies [1, 10, 15] operate at model-size granularity, co-varying format and size; none isolates the format at matched streamed bytes, and none reports the provenance sensitivity we find. Husom et al. [8] give the best-instrumented edge energy frontier across k-quant levels (Joulescope, Pi 4) and record without mechanism an energy inversion our model explains. Gope et al. [7] quantify unpack cost per format on Arm Cortex-A cores with heterogeneous scheduling; our work complements theirs with cross-ISA coverage analysis and the provenance sensitivity they do not consider. Chen et al. [3] survey quantization methods with deployment guidance but do not isolate kernel-level cost at matched bytes. T-MAC [14] and bitnet.cpp [13] identify dequantization as overhead and build lookup-table kernels; our contribution is the measurement complement: how large the kernel term is for the formats people already use, on which cores it binds, and what it costs in energy. Bit-width scaling laws [4] rank accuracy per byte; our results show the runtime cost side of that ranking is device-specific.

7 Limitations

Two model scales on both platforms, but perplexity was measured at 0.5B only, and the 1.5B runs on the 2 GB Pi execute under tight memory (about 1.3 GB available), so we report their ordering rather than absolute energy; the 1.5B quality frontier is the natural extension. Energy is measured on one board with uncalibrated PMIC rail sums, so energy claims are relative. The attribution uses aggregate per-run counters; per-kernel profiling would apportion the instruction inflation to individual GGML ops. Perplexity chunks are modest (51K tokens); format deltas exceed paired noise but downstream-task confirmation is future work.

8 Conclusion

The byte-streaming model of on-device decode is a good envelope and a bad contract. What a GGUF file costs is set by its per-tensor layout, its runtime's kernel coverage for the local core, and only then by its bytes; the label captures none of the first two. The community's default format is off the efficiency envelope on every core we measured, same-label files differ by more than a third depending on how they were made, and the format ranking flips between core types shipping in the same laptop. For a field that increasingly benchmarks, ships, and standardizes around format names, the fix is cheap and the numbers say it is worth up to a third of throughput and nearly half the energy budget: treat formats as device-specific implementation choices, and measure them as such.

Acknowledgments

Measurements were performed on the author's own hardware. All artifacts, raw measurement records, and analysis scripts are publicly released.

References

[1] Maximilian Abstreiter, Sasu Tarkoma, and Roberto Morabito. 2025. Sometimes Painful but Promising: Feasibility and Trade-offs of On-Device Language Model Inference. *arXiv preprint arXiv:2503.09114* (2025).

[2] Zhen Bi, Xueshu Chen, Luoyang Sun, Yuhang Yao, Qing Shen, Jungang Lou, and Cheng Deng. 2026. RooflineBench: A Benchmarking Framework for On-Device LLMs via Roofline Analysis. *arXiv preprint arXiv:2602.11506* (2026).

[3] Yuzhi Chen, Xiaomin Li, Dehua Kong, Wenzhe Yin, Ran Gong, and Yibo Fan. 2026. Which Quantization Method Is Right for You? A Practical Guide for LLM Quantization. *arXiv preprint arXiv:2601.14277* (2026).

[4] Tim Dettmers and Luke Zettlemoyer. 2023. The Case for 4-bit Precision: k-bit Inference Scaling Laws. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

[5] Georgi Gerganov and contributors. 2023. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp.

[6] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. AI and Memory Wall. *IEEE Micro* 44, 3 (2024), 33–39.

[7] Dibakar Gope, Jesse Beu, Atul Kwatra, and Guy Boudoukh. 2025. Efficient On-Device Inference of Quantized Large Language Models Using Heterogeneous Cores. *arXiv preprint arXiv:2501.00032* (2025).

[8] Erik Johannes Husom, Arda Goknil, Merve Astekin, Lwin Khin Shar, Andre Kåsen, Sagar Sen, Benedikt Andreas Mithassel, and Ahmet Soylu. 2025. Sustainable LLM Inference for Edge AI: Evaluating Quantized LLMs for Energy Efficiency, Output Accuracy, and Inference Latency. *ACM Transactions on Internet of Things* (2025). DOI 10.1145/3767742.

[9] Manu Nicholas Jacob. 2026. ML Systems Lab: unified edge-inference measurement framework and campaign dataset. https://github.com/manunicholasjacob/ml-systems-lab. Zenodo DOI 10.5281/zenodo.21867055.

[10] Stefanos Laskaridis, Kleomenis Katevas, Lorenzo Minto, and Hamed Haddadi. 2024. MELTing Point: Mobile Evaluation of Language Transformers. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*.

[11] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently Scaling Transformer Inference. In *Proceedings of Machine Learning and Systems (MLSys)*.

[12] Qwen Team. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).

[13] Jinheng Wang, Hansong Zhou, Ting Song, Shaoguang Mao, Shuming Ma, Hongyu Wang, Yan Xia, and Furu Wei. 2024. 1-bit AI Infra: Part 1.1, Fast and Lossless BitNet b1.58 Inference on CPUs. *arXiv preprint arXiv:2410.16144* (2024).

[14] Jianyu Wei, Shijie Cao, Ting Cao, Lingxiao Ma, Lei Wang, Yanyong Zhang, and Mao Yang. 2025. T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge. In *Proceedings of the 20th European Conference on Computer Systems (EuroSys)*.

[15] Jie Xiao, Qianyi Huang, Xu Chen, and Chen Tian. 2024. Understanding Large Language Models in Your Pockets: Performance Study on COTS Mobile Devices. *arXiv preprint arXiv:2410.03613* (2024).

[16] Zhihang Yuan, Yuzhang Shang, Yang Zhou, Zhen Dong, Zhe Zhou, Chenhao Xue, Bingzhe Wu, Zhikai Li, Qingyi Gu, Yong Jae Lee, Yan

Yan, Beidi Chen, Guangyu Sun, and Kurt Keutzer. 2024. LLM Inference Unveiled: Survey and Roofline Model Insights. *arXiv preprint arXiv:2402.16363* (2024).